

لماذا نبدأ بلغة C++

لأنها تعطيك مفاهيم البرمجة كلها ، لتستطيع تعلم أي لغة برمجة أخرى في أقل من شهرين

ماهي هي لغة C++ ومزاياها

- لغة متعددة الاستخدام تنشئ تطبيقات عالية الجودة يمكن تكييفها مع منصات مختلفة
- لغة ممتعة وسهلة
- هي أم اللغات
- يدعم البرمجة الهيكلية (Procedural) & OOP
- ستجبرك على تعلم مفاهيم البرمجة الأساسية
- إذا تعلمتها جيدا يمكنك تعلم أي لغة برمجة أخرى في أقل من شهر
- تعتبر لغة متوسطة المستوى قريبة من الأجهزة وقادرة على برمجة معدات الأجهزة الأساسية ومعالجتها
- لها تحكم كامل في موارد أو معدات الكمبيوتر مثل الذاكرة ،
- غير تلقائي إدارة الذاكرة

الدرس الرابع عشر : لماذا يجب أن تبدأ بتعلم C++

لأن هذه اللغة تأسسك بشكل قوي وتجبرك بتعلم مفاهيم البرمجة كلها
إذا تعلمت لغة C++ تستطيع تعلم أي لغة برمجة في أقل من شهر

ما هي لغة C++ ؟

هي لغة متعددة المنصات ، يمكن إنشاء تطبيقات عالية الأداء على أنظمة مختلفة

(cross-platform\Portable language)

ممتعة وسهلة التعلم

هي أم اللغات، لأنها بشكل مباشر أو غير مباشر مبنية على C++

تعد البرمجة الهيكلية و OOP

ستجبرك على تعلم مفاهيم البرمجة الأساسية

تستطيع تعلم أي لغة برمجة في أقل من شهر

تتمتع C, C++ بالتحكم الكامل في موارد (الهارد وير) الكمبيوتر مثل الذاكرة ، ولا توجد إدارة تلقائية للذاكرة

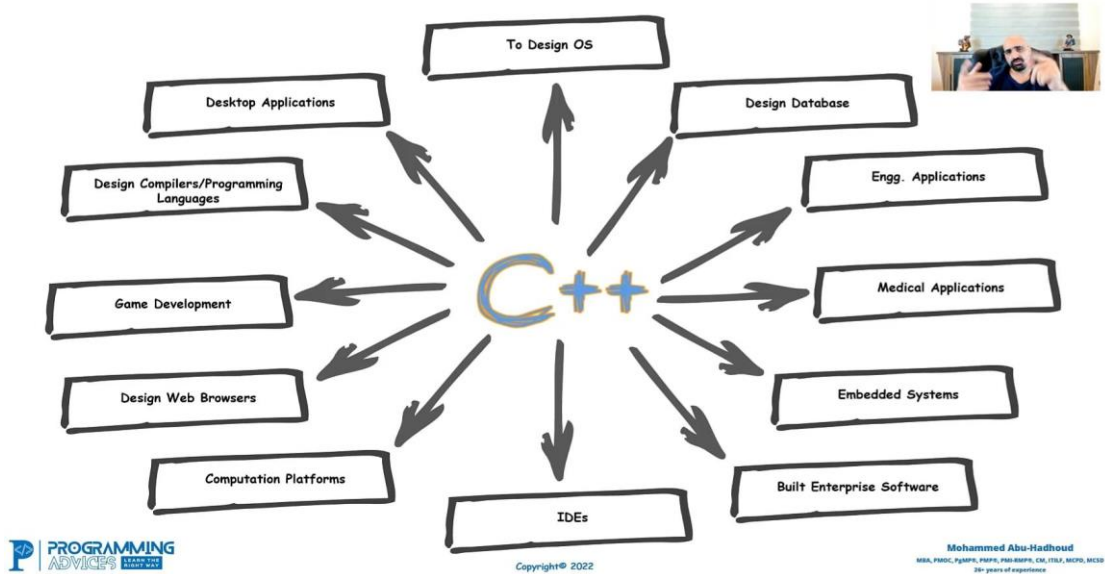
ما الذي تستطيع عمله بلغة C++؟

٣. لغات برمجة

٢. تطبيقات مثل
الفوتوشوب

١. أنظمة تشغيل

٤. قواعد بيانات	٥. مخططات هندسية	٦. ألعاب ضخمة
٧. برامج الأجهزة الطبية	٨. متصفح انترنت	٩. الأنظمة المضمنة مثل تسلا
١٠. البرامج الحاسوبية	١١. برامج ضخمة	١٢. IDEs فيجول استديو



المكتبات

عندما نحتاج الى حسابات رياضية عالية المستوى ، يصبح الأداء والسرعة مهمين

لذا فإن معظم المكتبات تستخدم C++ كلغة برمجة أساسية ،

تستخدم معظم مكتبات لغات High-level لغة C++ كخلفية لها

تعد لغة C++ أسرع من معظم لغات البرمجة الأخرى كما أنه يدعم تعدد مؤشرات الترابط مع التزامن

مميزات C++

- أداء ، سرعة
- قابلة للتكيف متعددة المنصات ، يمكن إنشاء تطبيقات عالية الأداء على أنظمة مختلفة
- تعدد العمليات الحسابية (معقدة ، كثيرة)
- عالية المستوى مع التزامن

مقارنة بين C++ و C

أوجه التشابه

- ❖ نفس syntax قواعد الكتابة أو بناء الجملة وكتابة الكود
- ❖ تستخدم لغة ترجمة Compiler
- ❖ كلاهما لغات متوسطة المستوى ، قريبة من الهارد وير
- ❖ التحكم الكامل في موارد (الهارد وير) الكمبيوتر مثل الذاكرة

أوجه الاختلاف

C++	C
أنشأت عام 1979	أنشأت بين 1969 الى 1973
لغة C++ أشمل من C	لغة C جزء من C++
برمجة هيكلية و OOP	برمجة هيكلية فقط (Procedural)

مقارنة بين C++ و Java

C++	Java
برمجة هيكلية و OOP	برمجة OOP فقط

قواعد كتابة اللغة (SYNTAX)

(#) لاستدعاء المكتبات مثل `#include <iostream>`

`Int main(){} لا بد من توفرها في لغة C++ {الأوامر}`

كل سطر لازم ينتهي ب فاصلة منقوطة ; ، لينتقل الى السطر الذي بعده

كيفية الطباعة (PRINT)

أمر الطباعة هو (std::cout << "Saeed";)

للنزول الى سطر جديد (std::cout << "Saeed\n") \n

أو (std::cout << "Saeed" << std::endl;)

لطباعة اكثر من امر بسطر واحد (std::cout << "Saeed" << "Omar";)

اختصار ل (STD::)

```
#include <iostream>
using namespace std;
int main()
{
    (std::cout << ("Saeed");) == (cout << "Saeed");
    std::cout << ("Saeed" << std::endl;) (cout << "Saeed" << endl;);
    return 0; }
```

الفرق بين \n & endl ؟

للنزول الى سطر جديد (std::cout << "Saeed\n") \n

أو (std::cout << "Saeed" << std::endl;)

الفرق بينهما في أوامر الطباعة الكثيرة ، التخزين المؤقت (Buffer)

\n يطبع الأوامر من غير تفريغ التخزين المؤقت يصبح البرنامج أبطئ

endl يطبع الأوامر مع تفريغ التخزين المؤقت اذا استخدمتها مع كل أمر طباعة يصبح البرنامج أبطئ لأنه يفرغ التخزين المؤقت مع كل أمر

لتسريع البرنامج استخدم \n مثلا كل ٥٠ أمر ثم استخدم واحد endl

endl ليست دائما أسرع ، لان تفريغ Buffer يأخذ وقت ، لذلك لا نستطيع أن نقول أن أحدهما أسرع من الآخر ، ولذلك تجب الموازنة بينهم

الملاحظات (COMMENTS)

يمكن استخدام الملاحظات لشرح الكود ولجعله أكثر قابلية للقراءة
كلما قلت الملاحظات كلما كان البرنامج أسرع (اكتب الملاحظات المهمة)
يمكن أن تكون التعليقات مفردة السطر أو متعددة الأسطر

١. مفردة السطر (Single line Comments) استخدم // أقل من سطر
٢. متعددة الأسطر (Multi line Comments) تبدأ ب /* وتنتهي ب /*

الأحرف (LITERALS)

البيانات الحرفية هي البيانات المستخدمة لتمثيل القيم - للمتغير - الثابتة،
أنواع الحروف

١. عدد صحيح حرفي
 - النظام العشري decimal (base 10)
 - النظام الثماني octal (base 8) 0
 - النظام السادس عشر hexadecimal (base 16) 0x
٢. الأعداد العشرية ١٢,٥ ٨,٣٣-
٣. الأحرف والرموز ABC, #, \$, %
٤. الاختصارات \ (Escape Sequences) مثل
 - \\\ للتعليقات
 - \t مسافة أربع فراغات
 - \n للنزول لسطر جديد
 - \", \' أقواس داخل أقواس للطباعة
 - \a صوت جرس

هي مكان لتخزين بيانات في الذاكرة

أنواع البيانات في C++

١. المتغيرات الأساسية (Fundamentals)

- الأرقام (integral Type)
 - رقم صحيح (int) = ٧١٢
 - حرف واحد (char) = "A"
- الأرقام الكسرية (Floating Type)
 - رقم كسري (float) = ١٤,٥
 - رقم كسري حجم أكبر (double)
- القيم المنطقية (Boolean Type) (bool) = True"1" OR False"0"
- النصوص (String Type) (string) = "Saeed"
- فارغ (Void Type) (void) = فارغة Empty
- جميع الأحرف (Wide Charater) (wchar_t) = أحرف عربية

٢. مشتق (Derived)

- Function
- Array
- Pointer
- Reference

٣. مستخدم محدد (User-Defined)

- Class
- Structure
- Union
- Enum
- Type Def

النوع	= اسم المتغير	القيمة
Type	Variable Name =	Value ;
int	MyAge =	45;
Cout << "My Age is:" << MyAge << endl ;		للطباعة

```

int myage = 22;
float MyFloatNumber = 32.54;
double MyDoubleNumber = 7.987657;
char MyLetter = 'S';
string MyText = "Seed";
bool MyBoolean = true;

cout << "My Age is:" << myage << endl;
cout << myage << endl;
cout << MyFloatNumber << endl;
cout << MyDoubleNumber << endl;
cout << MyLetter << endl;
cout << MyText << endl;
cout << MyBoolean << endl;

```

تسمية المتغيرات

۵. یوصی باستخدام الاسماء الوصفیه لإنشاء كود مفهوم ويمكن صيانته

معرفات

يجب تحديد جميع متغيرات ++C بأسماء فريدة.

المعرف هو اسم المتغير.

تسمى هذه الأسماء الفريدة بالمعرفات.

يمكن أن تكون المعرفات أسماء قصيرة (مثل x و y) أو أسماء وصفية أكثر (العمر ، المجموع ، إجمالي المبيعات).

ملاحظة: يوصى باستخدام الأسماء الوصفية لإنشاء كود مفهوم

يمكن صيانتها



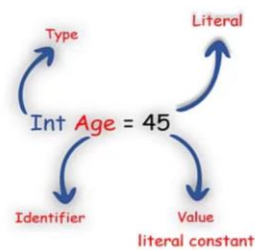
All C++ variables must be identified with unique names.

The IDENTIFIER is the name of the variable.

These unique names are called identifiers.

Identifiers can be short names (like x and y) or more descriptive names (Age, Sum, TotalSales).

Note: It is recommended to use descriptive names in order to create understandable and maintainable code



القواعد العامة لتسمية المتغيرات (VARIABLES) والثوابت (CONSTANTS) هي

١. يمكن أن تحتوي الأسماء على أحرف وأرقام وشرطات سفلية
٢. يجب أن تبدأ الأسماء بحرف أو شرطة سفلية
٣. الأسماء حساسة لحالة الحرف (myAge و myage متغيرين مختلفين)
٤. لا يمكن أن تحتوي الأسماء على مسافات أو رموز خاصة مثل \$%^&*
٥. لا يمكن استخدام الكلمات المحجوزة مثل int , cout , string

Keywords				
alignas	alignof	and	and_eq	asm
auto	bitand	bitor	bool	break
case	catch	char	char16_t	char32_t
class	compl	const	const_cast	constexpr
continue	decltype	default	delete	do
double	dynamic_cast	else	enum	explicit
export	extern	false	float	for
friend	goto	if	inline	int
long	mutable	namespace	new	noexcept
not	not_eq	nullptr	operator	or
or_eq	private	protected	public	register
reinterpret_cast	return	short	signed	sizeof
static	static_assert	static_cast	struct	switch
template	this	thread_local	throw	true
try	typedef	typeid	typename	union
unsigned	using	virtual	void	volatile
wchar_t	while	xor	xor_eq	

والثوابت (CONSTANTS) هي قيمة غير قابلة للتغيير، هي للقراءة فقط

القيمة	اسم الثابت	النوع	تثبيت
3.14	PI =	float	const
60	minutesPerHour =	int	const

هل أسماء المتغيرات التالية صالحة ولماذا ؟

Age	نعم
My-Age	لا ، لأن الكمبيوتر يفهمها معادلة
My_Age	نعم
_Age	نعم
2x	لا البداية رقم
X2	نعم
My Name	لا لوجود المسافة
Myname	نعم
#Name	لا لوجود الرمز
&Name	لا لوجود الرمز
N@me	لا لوجود الرمز
Int	لا لوجود اسم خاص ب C++
float	لا لوجود اسم خاص ب C++

الإدخال (INPUT)

قواعد كتابة الإدخال Input Syntax

يتم استخدام cin مع عامل التشغيل >> أكبر من لإدخال القيم الى المتغيرات

Std::cin

>>

Variable;

أحجام أنواع البيانات (DATA TYPE) لكل نوع له حجم مناسب

Ranges & Sizes				
Type	Meaning	Range		Size (Bytes)
int	Integer	-2,147,483,648	+2,147,483,647	4
float	Floating-Point	1.17549e-38	3.40282e+38	4
double	Double Floating-Point	2.22507e-308	1.79769e+308	8
char	Character	-127	127	1
wchar_t	Wide Character			2
bool	Boolean	0	1	1
void	Empty			0
string	String			12

& معدل النوع (TYPE MODIFIERS)

يمكننا تقليص أحجام بعض أنواع البيانات الأساسية باستخدام معدلات النوع ، توجد أربع ٤ معدلات من النوع في C++ وهي :

١. Signed (+ و -)
٢. Unsigned (+)
٣. Short (قصير)
٤. Long (كبير)

Ranges & Sizes				
Type	Sign	Range		Size
short int short	signed	-32,768	+32,767	2bytes
	unsigned	0	65,536	2bytes
int	signed	-2,147,483,648	+2,147,483,647	4 bytes
	unsigned	0	4,294,967,295	4 bytes
long int long	signed	-2,147,483,648	+2,147,483,647	4 bytes
	unsigned	0	4,294,967,295	4 bytes
long long int long long	signed	-(2 ⁶³)	(2 ⁶³)-1	8 bytes
	unsigned	0	18,446,744,073,709,551,615	8 bytes

هذه المعدلات الأربع ٤ تستخدم لأنواع البيانات التالية

- ١. Int
- ٢. Double
- ٣. Char

Ranges & Sizes				
Type	Sign	Range		Size
float	signed	1.17549e-38	3.40282e+38	4 bytes
double	signed	2.22507e-308	1.79769e+308	8 bytes
long double	signed	10 e-307	10 e+308	12 bytes

Ranges & Sizes				
Type	Sign	Range		Size
char	signed	-127	127	1 byte
	unsigned	0	255	1 byte
bool	-			1 byte
String	-			12 bytes
Void	-	Empty		0 bytes

$$10 * 10^{308} = 10e+308^*$$

Short d = 3434233 ; خطأ لأن أكبر حجم short هو = 32767 و -32768

وإذا كان unsigned short = 65536

لمعرفة أحجام أنواع البيانات

```

7 int main()
8 {
9     cout << "*****\n";
10    cout << "char Range: (" << CHAR_MIN << ", " << CHAR_MAX << ")\n";
11    cout << "unsigned char Range: (" << 0 << ", " << UCHAR_MAX << ")\n";
12
13    cout << "short int Range: (" << SHRT_MIN << ", " << SHRT_MAX << ")\n";
14    cout << "unsigned short int Range: (" << 0 << ", " << USHRT_MAX << ")\n\n";
15
16    cout << "int Range: (" << INT_MIN << ", " << INT_MAX << ")\n";
17    cout << "unsigned int Range: (" << 0 << ", " << UINT_MAX << ")\n";
18    cout << "long int Range: (" << LONG_MIN << ", " << LONG_MAX << ")\n";
19    cout << "unsigned long int Range: (" << 0 << ", " << ULONG_MAX << ")\n\n";
20
21    cout << "long long int Range: (" << LLONG_MIN << ", " << LLONG_MAX << ")\n";
22    cout << "unsigned long long int Range: (" << 0 << ", " << ULLONG_MAX << ")\n\n";
23
24    cout << "float Range: (" << FLT_MIN << ", " << FLT_MAX << ")\n";
25    cout << "float(negative) Range: (" << -FLT_MIN << ", " << -FLT_MAX << ")\n\n";
26
27    cout << "double Range: (" << DBL_MIN << ", " << DBL_MAX << ")\n";
28    cout << "double(negative) Range: (" << -DBL_MIN << ", " << -DBL_MAX << ")\n";
29    cout << "long double Range: (" << LDBL_MIN_10_EXP << ", " << LDBL_MAX_10_EXP << ")\n";
30    cout << "*****\n";
31    return 0;

```

العوامل الحسابية

+ جمع ، - ناقص ، ضرب * ، قسمة / ، باقي القسمة %

عوامل التشغيل الزيادة (INCREMENT) ++ ، والنقصان (DECREMENT) --

طريقة الكتابة	الناتج	A = 10 , B = 20	Operator
A++; ++A;	١١	إذا كانت في A++ (A=A+1) سطر بمفردها ++A (A=A+1)	++
B++; ++B;	١٩	لا يوجد B-- (B=B-1) فرق بينهما --B (B=B-1)	--

الفرق بين (INCREMENT) ++ ، والنقصان (DECREMENT) --

يكون الفرق إذا لم تكن بمفردها مثل c=a++, and c=++a

طريقة الكتابة	الناتج	مثال A = 10	Operator
Int A = 10; Int C = A++;	C سيأخذ A = 10 ثم تزيد A + 1 = 11	C = A++	A++
int A = 10; Int C = ++A	A ستزيد 1 = 11 ثم C ستأخذ قيمة A = 11	C = ++A	++A
Int A = 10; Int C = A--;	C سيأخذ A = 10 ثم تنقص A - 1 = 9	C = A--	A--
int A = 10; Int C = --A	A تنقص 1 = 9 ثم C ستأخذ قيمة A = 9	C = --A	--A

عمليات المقارنة (RELATIONAL OPERATORS)

Int A = 10 , B =20;

Operator	معنى العامل	مثال	الجواب
==	التحقق مما إذا كانت قيم العاملين متساوية أو لا	Cout << (A == B) << endl;	0
!=	التحقق مما إذا كانت قيم العاملين غير متساوية أم لا	Cout << (A != B) << endl;	1
<	العامل الأيمن أكبر من العامل الأيسر	Cout << (A < B) << endl;	1
>	العامل الأيمن أصغر من العامل الأيسر	Cout << (A > B) << endl;	0
<=	العامل الأيمن أكبر من العامل الأيسر أو يساوي	Cout << (A <= B) << endl;	1
>=	العامل الأيمن أصغر من العامل الأيسر أو يساوي	Cout << (A >= B) << endl;	0

العوامل المنطقية (LOGICAL OPERATORS)

Bool A = 1 (Ture) , B = 0 (false);

العامل	المعنى	مثال	الجواب
&&	إذا كان الشرطين صحيحة AND	Cout << (A && B) ;	0
	إذا كان احد الشرطين صحيح OR	Cout << (A B) ;	1
!	لنفي الشرط NOT	Cout << !(A && () B) ; Cout << !A;	1 (0) 0

Functions هو اختصار للكود (بدل كتابة عدة أسطر استخدم كود جاهز في أقل من سطر)

```
#include <iostream>
#include <cmath>
using namespace std;
int main()
{
    ...
}
```

الاسم	Name	اختصار	طريقة الكتابة
الجذر	Squair Root	sqrt	Cout << sqrt(9);
التقريب	Rounding	round	Cout << round(3.7);= 4 ^١
الأس	Power	pow	Cout << pow(2,3); = 8
الأعلى ^٢ (السقف)	ceil	ceil	Cout << ceil(2,1); = 3
الأدنى ^٣ (الأرض)	floor	floor	Cout << floor(2,9); = 2
القيمة المطلقة	Absolute	abs	Cout << abs(-10); = 10

...<https://en.cppreference.com/w/cpp/hea>

١ التقريب من ٥ وأعلى يعطي رقم أكبر ، ومن ٤ وأسفل يعطي نفس الرقم صحيح

٢ الأعلى ceil يقرب دائما الى الرقم الأعلى مثال : ٢,١ = ٣ / ٤,٦ = -٥

٣ الأدنى floor يقرب دائما الى أسفل مثال : ٢,٩ = ٢ / ٨,٦ = -٦

الهياكل (STRUCTURES)

مستخدم محدد (User-Defined)

- Class
- Structure
- Union
- Enum
- Type Def

ما هو الهيكل (Structures)

طريقة لتجميع العديد من المتغيرات ذات صلة في مكان واحد ، يعرف كل متغير في الهيكل بأنه عضو في بنية الهيكل ، (يمكن بناء هيكل داخل هيكل)

```
# include <iostream>
using namespace std;
struct car
{
    string Brand;
    string Model;
    int Years;
};
int main()
{
    car Mycar1, Mycar2;
    Mycar1.Brand = "BMW";
    Mycar1.Model = "X5";
    Mycar1.Years = 2000;
    Mycar2.Brand = "Ford";
    cout << Mycar1.Brand << " " << Mycar1.Model << " " << Mycar1.Years
    << "\n";
}
```


مستخدم محدد (User-Defined)

- Class
- Structure
- Union
- Enum
- Type Def

التعداد هو : نوع مميز تقتصر قيمته على نطاق من القيم (تبدأ أول قيمة من الصفر تلقائياً ، أو تضع قيمة لها)

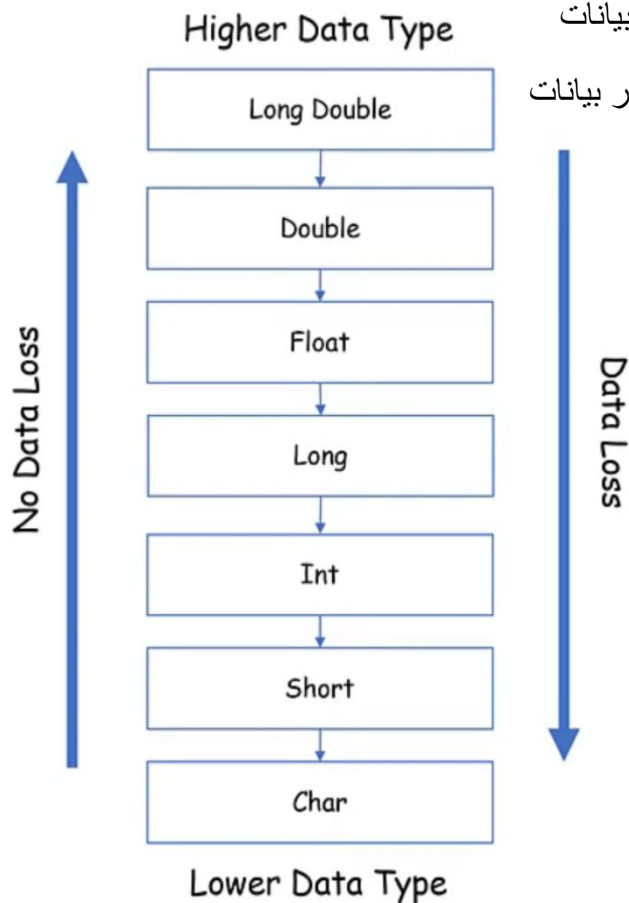
```
# include <iostream>
using namespace std;
enum Week{ Sat , Sun , Mon , Tue , Wed , Thu , Fri };
int main()
{
    Week Today;
    Today = Week::Sat;
    cout << Today;
}
```

تحويل نوع البيانات (DATA TYPE CONVERSION)

تحويل من	تحويل الى	طريقة التحويل	مثال
double	int	Num1 = Num2; ^٤	int Num1; double Num2 = 18.99;
		Num1 = (int) Num2	
		Num1 = int(Num2)	
String	int	int Num_int = stoi(str);	#include <string>
	float	float Num_float = stof(str);	
	double	double Num_double = stod(str);	string str = "123.456";
Number	string	st1 = to_string(Num1);	#include <string> int num1 = 123
		st2 = to_string(Num2);	double num2 = 11.65; string st1, st2;

تحويل البيانات من الكبير الى الصغير تخسر بيانات

تحويل البيانات من الصغير الى الكبير لا تخسر بيانات



Copyright© 2022

° اختصار ل (int, double, float) string to

هو: مجموعة من الأحرف محاطة بعلامات اقتباس مزدوجة

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string mystring = "ABCDEFGHJKLMNOPQRSTUVWXYZ";
    // طباعة عدد الأحرف
    cout << mystring.length() << "\n"; // print 26

    // طباعة حرف من الأحرف (يبدأ أول حرف من رقم 0)
    cout << mystring[2] << "\n"; // print "C"

    // string الأرقام في هي نص وليست رقم
    string S1 = "10", S2 = "20";
    string S3 = S1 + S2;
    cout << S3 << "\n"; // print "1020"

    // int الى string لابد من تحويل string لجمع الأرقام في
    // لجمع الأرقام في النصوص لابد من تحويل النصوص الى رقم
    int sum = stoi(S1) + stoi(S2);
    cout << sum << "\n"; // print 30

    // (saeed omar) الاسم كامل بعد الفراغ cin مشكلة عدم طباعة النص
    string FullName;
    cin >> FullName; // (saeed omar maroon)
    cout << FullName << "\n"; // print saeed .....
    // (saeed omar) الاسم كامل بعد الفراغ cin حل مشكلة عدم طباعة النص
    // استخدم أمر قبل enter عند الضغط على (...) Getline(cin, ...) حل مشكلة تجاوز إدخال
    cin.ignore(1, "\n");
    string FullName;
    cin.ignore(1, '\n');
    getline(cin, FullName); // (saeed omar amroon)}
}
```

الوظائف Functions هي : عبارة عن كتلة من التعليمات البرمجية يتم تشغيلها فقط عند استدعائها (اسمها في (int main()

ووظيفته فصل جزء من الكود لإعادة استخدامه أكثر من مرة

يمكنك تمرير البيانات المعروفة باسم المعلمة الى دالة

تستخدم الدوال لأداء إجراءات معينة وهي مهمة لإعادة استخدام الكود ، حدد الرمز مرة واحدة واستخدمه عدة مرات

الفرق بين Functions و procedure

Functions يرجع قيمة (مثل المعادلات) لا يقرأ cin ولا يطبع cout

procedure هو Functions لكن لا يرجع قيمة (مثل أوامر الطباعة – فارغ Void)

مميزات Functions

- سهولة كتابة الكود
- تجزئة كتابة الكود
- سهولة فحص الكود
- Functions دوال غير محدودة
- الأفضل في Functions الدالة أن تعمل شيء واحد

الاختصار	Name	الاسم
sqrt	Squair Root	الجذر
round	Rounding	التقريب
pow	Power	الأس
ceil	ceil	الأعلى ^٦ (السقف)
floor	floor	الأدنى ^٧ (الأرض)
abs	Absolute	القيمة المطلقة

أمثلة على Functions الجاهزة

مكتبة الرياضيات (math Functions)

^٦ الأعلى ceil يقرب دائما الى الرقم الأعلى مثال : $2.1 = 3 / 6.4 = -5$

^٧ الأدنى floor يقرب دائما الى أسفل مثال : $2.9 = 2 / 6.8 = -6$

```

#include <iostream>
#include <string>
using namespace std;
//Void مثل أوامر الطباعة - فارغ) لكن لا يرجع قيمة Functions هو
void MyProcedure()
{
    cout << "This is my function , it got executed " << endl;
}
//Functions التي ترجع قيمة واحدة فقط يستدعى مع Return يرجع قيمة (مثل المعادلات) لا بد من استخدام
cout <<
string MyFunction1()
{
    return "This is my first returning value function , this is value ";
}
float myFunction2()
{
    float x = 10.8;
    float y = 20.4;
    return x* y;
}
//Functions cout ولا يطبع cin يرجع قيمة (مثل المعادلات) لا يقرأ
int myFunction3(int Num1, int Num2)
//Parameters هذه تسمى
{
    return Num1 + Num2;
}

int main()
{
    MyProcedure();
    cout << MyFunction1();
    float Result;
    Result = round(myFunction2());
    cout << Result;

    cout << myFunction3(10, 20) << endl;
    //function وليس في int main إذا تريد أن يدخل المستخدم بيانات ، تنفذ هذا في
    int Num1, Num2;
    cin >> Num1;
    cin >> Num2;
    cout << myFunction3(Num1, Num2) << endl;
}

```

دورة حياة المتغير (VARIABLE SCOPE) { مكان نطاق المتغير }

لكل متغير: اسم وقيمة وعنوان (موقعه في الذاكرة)

```
#include <iostream>
#include <cmath>
#include <string>
using namespace std;
// Function أي تستطيع استخدامه داخل أي دالة (Global)النطاق يسمى : نطاق عالمي
int x = 300;

// {} يكون داخل أقواس (Local)هذا النطاق يسمى : نطاق محلي
void myProcedure()
{
    int x = 500;
    cout << "The value of x inside function is : " << x << endl;
}

// (Local)هذا النطاق يسمى : نطاق محلي
int main()
{
    int x = 10;
    cout << "The local value of x inside main is : " << x << endl;
    myProcedure();

    // variable يكون بنقطتين رأسيتين: ثم اسم المتغير Global طريقة استدعاء النطاق العالمي
    cout << "The Global value of x is : " << ::x << endl;

    // ++طريقة زيادة رقم واحد على المتغير تكون ب
    ::x++; //print 301

    // طريقة تغيير أي متغير تكون باستدعائه ثم = ووضع القيمة الجديدة
    ::x = 7000;
}
```

```

#include <iostream>
using namespace std;
// By value تستطيع إعطاء نسخة من الأصلية والتعديل عليها (تستطيع إعطاء نسخة من الأصلية والتعديل عليها)
// By value هي نسخة من Num1 الموجودة في main (تستطيع إعطاء نسخة من الأصلية والتعديل عليها)
// فقط function داخل
void MyProcedure1(int Num1)
{
    Num1 = 7000;
    cout << "Number inside function became = " << Num1 << endl;
}
// By Reference أي يعطي القيمة (أي يعطي القيمة) & باستخدام علامة int main أي مرجع هي العنوان للمتغير الأصلي في main (أي يعطي القيمة)
// باستخدام علامة int main أي مرجع هي العنوان للمتغير الأصلي في main (أي يعطي القيمة)
// hexadecimal عنوانه هو hexadecimal عنوانه هو hexadecimal
// hexadecimal عنوانه هو hexadecimal عنوانه هو hexadecimal
void MyProcedure2(int& Num2)
{
    Num2 = 9000;
    cout << "Number inside function became = " << Num1 << endl;
}
int main()
{
    int Num1;
    Num1 = 1000;
    MyProcedure1(Num1);
    cout << "Number after calling the function became = " << Num1 <<
endl;
    // print 1. MyProcedure1 = 7000 , 2. Num1 = 1000

    int Num2;
    Num2 = 3000;
    MyProcedure2(Num2);
    cout << "Number after calling the function became = " << Num1 <<
endl;
    // print 1. MyProcedure2 = 9000 , 2. Num2 = 3000
    // لطباعة عنوان المتغير Hexadecimal
    cout << &Num2;
}

```

كلما قل الكود كلما كانت الأخطاء أقل وكلما كانت البرمجة أمتع

```
#include <iostream>
#include <string>

using namespace std;
struct strInfo
{
    string FullName;
    short Age;
    string Phone;
};
void ReadInfo(strInfo& Info)
{
    cout << "Pleas enter Full Name : ";
    getline(cin, Info.FullName);
    cout << "Pleas enter Age : ";
    cin >> Info.Age;
    cout << "Pleas enter Phone : ";
    cin >> Info.Phone;
}
void PrintInfo(strInfo Info)
{
    cout << "*****\n";
    cout << "Full Name : " << Info.FullName << endl;
    cout << "Age : " << Info.Age << endl;
    cout << "Phone : " << Info.Phone << endl;
    cout << "*****\n";
}
int main()
{
    strInfo Person1Info;
    ReadInfo(Person1Info);
    PrintInfo(Person1Info);
}
```


الصفائف (Arrays) هي : متغير تستطيع تخزين أكثر من قيمة بشرط أن يكون من نفس نوع البيانات (int , string , float) يبدأ أول عنصر من الفهرس index من 0

```
#include <iostream>
using namespace std;
```

```
int main()
{
```

```
    // يعني أن المصفوفة خزنت خمسة (عناصر) قيم من نوع واحد يبدأ من رقم واحد x[5]
    int x[8] = { 22, 33, 6, 5 , 7 };
```

```
    // تبدأ عناصر المصفوفة أي أول قيمة من index[0] دائما فهرس = العنصر ناقص 1
    // كل عنصر هو قيمة لكن تحت اسم متغير واحد تستطيع جمعها أو قسمتها بأي قيمة في متغير آخر
    x[5] = 11;
    x[6] = 44;
    x[7] = 4;
```

```
    cout << x[0] << endl;
    cout << x[1] << endl;
    cout << x[2] << endl;
    cout << x[3] << endl;
    cout << x[4] << endl;
    cout << x[5] << endl;
    cout << x[6] << endl;
    cout << x[7] << endl;
```

```
}
```

الصفائف مع الدوال (ARRAYS WITH FUNCTION)

هي : متغير تستطيع تخزين أكثر من قيمة بشرط أن يكون من نفس نوع البيانات (int , string , float) يبدأ أول عنصر من الفهرس index من 0

```
#include <iostream>
using namespace std;
// فقط معرف مسبقا ب Array لأن &Ref - By لا يحتاج الى Array في Parameters مهم هذا
void ReadArrayNumber(int Num[3])
{
    cout << "pleas enter Number1 ";
    cin >> Num[0];
}
void PrintArrayNumber(int Num[3])
{
    cout << Num[0];
}
int main()
{
    // int (كأنك عرفت 5 متغيرات) من int أي: تحجز 5 أماكن من الذاكرة من النوع
    int x[5] = { 43 ,55, 2, 67,90 };
    // index الفهرس x ، اسم المتغير x لطباعة/قيمة من المتغير

    cout << x[0] << endl; // print 43

    //تستطيع طلب إدخال البيانات من المستخدم
    int y[5];
    cin >> y[0]; // [1] , [2] , [3] , [4]
    x[4] = 676;
    int Num[3];
    // باسم المتغير فقط ، بدون الأقواس يكون

    ReadArrayNumber(Num);
    PrintArrayNumber(Num);
}
```

الهياكل مع الدوال (ARRAYS OF STRUCTURE)

```
#include <iostream>
using namespace std;

struct strInfo
{
    string Name;
    short Age;
    string Phone;
};
int main()
{
    strInfo Person[6];
    Person[0].Name = "Saeed";
    Person[2].Age = 22;
    Person[5].Phone = "0987765";
}
```

```

#include <iostream>

using namespace std;
int main()
{
    int Time = 10;
    int x;
    cin >> x;

    if (x > 5 && x < 20)
    {
        if (Time < 10)
        {
            // {} ينفذ بين القوسين (إذا تحقق الشرط = صحيح)
            cout << "Good Morning.";
        }
    }

    // يمكن استخدامه بعدد لا نهائي else if إذا كان يوجد أكثر من شرط نستخدم
    else if (Time < 20)
    {
        // {} ينفذ بين القوسين (إذا تحقق الشرط = صحيح)
        cout << "Good day ";
    }

    else
    {
        // {} نفذ بين القوسين (خطأ = false) إذا لم يتحقق الشرط أو الشروط
        cout << "Good evening ";
    }
}

```

العداد مع الجمل الشرطية (ENUMS WITH IF)

```
#include <iostream>
using namespace std;

enum enCountryChoice { Jordan = 1, Tunisia = 2, Algeria = 3, Oman = 4,
Egypt = 5, Iraq = 6, Other = 7 };

int main()
{
    cout << "*****\n";
    cout << "Please Enter the number of your country?\n";
    cout << "(1) Jordan\n";
    cout << "(2) Tunisia\n";
    cout << "(3) Algeria\n";
    cout << "(4) Oman\n";
    cout << "(5) Egypt\n";
    cout << "(6) Iraq\n";
    cout << "(7) Other\n";
    cout << "*****\n\n";
    cout << "Your Choice? ";

    // enum عن طريق cin لا بد من إضافة متغير وتحويل المتغير الى enum مثل التحويل الى
    // طريقة قراءة
    int c;
    enCountryChoice Coutnry;

    cin >> c;
    Coutnry = (enCountryChoice)c;

    if (Coutnry == enCountryChoice::Jordan)
    {
        cout << "Your country is Jordan\n";
    }
    else if (Coutnry == enCountryChoice::Tunisia)
    {
        cout << "Your country is Tunisia\n";
    }
    else if (Coutnry == enCountryChoice::Algeria)
    {
        cout << "Your country is Algeria\n";
    }
    else if (Coutnry == enCountryChoice::Oman)
    {
        cout << "Your country is Oman\n";
    }
    else if (Coutnry == enCountryChoice::Egypt)
    {
        cout << "Your country is Egypt\n";
    }
    else if (Coutnry == enCountryChoice::Iraq)
    {
        cout << "Your country is Iraq\n";
    }
    else
    {
        cout << "Your country is Other\n";
    }

    return 0;}
```

```
int day;
cin >> day;
switch (day)
{
    // Case 1: Switch يعني إذا كان صحيح توقف واخرج من day == 1 / break
    case 1:
        cout << "Sunday";
        break;
    case 2:
        cout << "Monday";
        default case أي إذا كانت (تستطيع الاستغناء عنه)
        default:
            cout << "Not a weekday ";
}
```

```

#include <iostream>
using namespace std;

enum enWeekDay { Sun = 1, Mon = 2, Tue = 3, Wed = 4, Thu = 5, Fri = 6, Sat = 7
};
void ShowWeekDayMenu()
{
    cout << "*****" << endl;
    cout << "      Weekdays      " << endl;
    cout << "*****" << endl;
    cout << "1: Sunday" << endl;
    cout << "2: Monday" << endl;
    cout << "3: Tuesday" << endl;
    cout << "4: Wednesday" << endl;
    cout << "5: Thursday" << endl;
    cout << "6: Friday" << endl;
    cout << "7: Saturday" << endl;
    cout << "*****" << endl;
    cout << "Please enter the number of days?" << endl;
}

//enum من enum وتحويل الى int
enWeekDay ReadWeekDay()
{
    int wd;
    cin >> wd;
    return (enWeekDay)wd;
}

string GetWeekDayName(enWeekDay WeekDay)
{
    switch (WeekDay) {

        case enWeekDay::Sun:
            return "Sunday";
            break;
        case enWeekDay::Mon:
            return "Monday";
            break;
        case enWeekDay::Tue:
            return "Tuesday";
            break;
        case enWeekDay::Wed:
            return "Wednesday";
            break;
        case enWeekDay::Thu:
            return "Thursday";
            break;
        case enWeekDay::Fri:
            return "Friday";
            break;
        case enWeekDay::Sat:
            return "Saturday";
            break;
        default:
            return "Not a weekday!\n";
    }
}

int main()
{
    ShowWeekDayMenu();
    cout << "today is " << GetWeekDayName(ReadWeekDay()) << endl;
}

```

حلقات التكرار (FOR LOOPS)

تساعدك على تكرار كود معين عدد من المرات ، دون تكرار الكود

طريقة عمل حلقات التكرار: إذا كان الشرط صحيح تعمل حلقات التكرار وإذا كان خطأ يخرج من حلقات التكرار

```
#include <iostream>
using namespace std;
int main ()
{
    for (int i = 1; i <= 5; i++)
        //(int i = 1; هذه بداية \; i <= 5; هذا الشرط \ i++ هذا تحديث للشرط
    {
        cout << i << endl << "Saeed \n ";
    }
}
```

حلقات التكرار مع الصفائف (FOR LOOPS & ARRAY)

```
#include <iostream>
#include <string>
using namespace std;

struct strInfo
{
    string Name;
    int Age;
    string Phone;
};

void ReadInfo(strInfo &Info)
{
    cout << "Pleas enter Name :\n";
    cin.ignore(1, '\n');
    getline(cin, Info.Name);

    cout << "Pleas enter Age : \n";
    cin >> Info.Age;

    cout << "Pleas enter Phone : \n";
    cin >> Info.Phone;
}

void PrintInfo(strInfo Info)
{
    //cout << "*****\n";

    cout << "Name : " << Info.Name << endl;
    cout << "Age : " << Info.Age << endl;
    cout << "Phone : " << Info.Phone << endl;

    cout << "*****\n\n";
}

void ReadPersonInfo(strInfo Persons[100] , int& NumberofPersons)
{
    cout << "How Many Persons ? 1 to 100 ? \n";
    cin >> NumberofPersons;

    for (int i = 0; i <= NumberofPersons - 1; i++)
    {
        cout << "Pleas Enter Person's [" << i + 1 << "] Info: \n";
        ReadInfo(Persons[i]);
        cout << "\n*****\n";
    }
}

void PrintPersonInfo(strInfo Persons[100], int NumberofPersons)
{
    for (int i = 0; i <= NumberofPersons - 1; i++)
    {
        cout << "Person's [" << i + 1 << "] Info: \n";
        PrintInfo(Persons[i]);
    }
}

int main()
{
    strInfo Persons[100];
    int Length = 1;

    ReadPersonInfo(Persons, Length);
    PrintPersonInfo(Persons, Length);}
```


حلقات التكرار داخل بعض (NESTED FOR LOOPS)

```
for (int x = 1; x <= 10; x++)
{
    for (int y = x; y <= 10; y++)
    {
        cout << y << " ";
    }
    cout << "\n";
}
```

حلقات التكرار غير المحدودة (WHILE LOOPS)

تستخدم إذا كنت لا تعرف كم تريد تكرارها

طريقة عمل حلقات التكرار: إذا كان الشرط صحيح تعمل حلقات التكرار وإذا كان خطأ يخرج من حلقات التكرار

```
#include <iostream>
using namespace std;

int ReadIntNumberInRange(int Form, int To)
{
    int Number;
    cout << "Pleas enter a number between " << Form << " and " << To <<
endl;
    cin >> Number;

    // هل 18 أصغر من < الرقم أو || أكبر من > الرقم ( \\ الرقم 15
    // الجواب نعم 15 أصغر من <18 يعني أدخل في حلقة التكرار )

    while (Number < Form || Number > To)
    {
        cout << "Wrong Number,";
        cout << "Pleas enter a number between " << Form << " and " << To
<< endl;
        cin >> Number;
    }
    return Number;
}

int main()
{
    cout << "\n The Number is : " << ReadIntNumberInRange(18, 45) << endl;
}
```

حلقات التكرار غير المحدودة (DO WHILE LOOPS)

Do while loops هي نفذ ثم اعترض (ادفع ثم اعترض) \ هي اختصار للكود في حال التكرار

while loops هي عكس Do while loops أي اعترض ثم نفذ (اعترض ثم ادفع)

```
#include <iostream>
using namespace std;

int ReadIntNumberInRange(int Form, int To)
{
    int Number;

    // (اختصر قراءة الرقم ، بدل قراءة تكرار الكود أصبحت قراءة واحدة كود واحد)

    do
    {
        cout << "Pleas enter a number between " << Form << " and " << To
<< endl;
        cin >> Number;

    } while (Number < Form || Number > To);

    return Number;
}

int main()
{
    cout << "\n The Number is : " << ReadIntNumberInRange(18, 45) << endl;
}
```

توقف (BREAK)

تستخدم للخروج من حلقات التكرار سواء (For Loops Or While Loops)

الكود بعد Break لا ينفذ

```
for (int x = 1; x <= 10; x++)
{
    if (x == 5)
        // إذا كان الشرط صحيح اخرج خارج حلقة التكرار
        break;

    cout << x << endl; // Print 1,2,3,4,
}

int x = 1;
while ( x <= 10)
{
    if (x == 5)
        // إذا كان الشرط صحيح اخرج خارج حلقة التكرار
        break;

    cout << x << endl; // Print 1,2,3,4,
    x++;
}
```

أكمل (CONTINUE)

تستخدم للرجوع الى بداية حلقة التكرار سواء (For Loops Or While Loops) لتنفيذ الشرط

الكود بعد continue لا ينفذ

```
for (int x = 1 ; x <= 5 ; x++)
{
    if (x == 3)
    {
        // إذا تحقق الشرط (ارجع ) الى بداية حلقة التكرار
        continue;
        // لا تستمر الى نهاية حلقة التكرار
    }
    cout << x << endl; // Print 1 , 2 , 4 , 5
}
```